

ABM BUSINESS LOGIC BIBLE

SECTION 2 · TIER 6 — STRATEGIC MEMORY

From Cognitive Schema to Judgment: the entities that store WHY reality behaves as it does

Tiers 1-5 model State, Decision, Governance, and Analytics — they learn WHAT happened. Tier 6 models Character: persistent strategic identity, market narratives, regret, and inter-account dependency. These entities accumulate transferable judgment, marking the system's shift from an ABM engine to a Cognitive Enterprise Operating System. Four new entities, one edge decomposition, three tier-defining meta-principles — each excavated via full 5-Whys.

New entities	4 (+1 decomposition)	5-Whys	20
New rules	26	Meta-principles	3

TIER 6 — STRATEGIC MEMORY ENTITIES

The entities that store WHY reality behaves the way it does — persistent across signals, windows, and leadership. Tiers 1-5 learn what happened; Tier 6 accumulates transferable judgment. Admission test for this tier: an entity belongs here only if it makes the system WISER (judges better next time, even on a different account), not merely BUSIER (records more activity).

Tier-Defining Meta-Principles

Unlike operational rules, these are immutable principles that DEFINE what qualifies as Strategic Memory. The admission test (META-T6-001) is the lens applied to every candidate entity: does it make the system wiser, not merely busier?

META-PRINCIPLE — META-T6-001: TIER 6 ADMISSION TEST: An entity is Strategic Memory only if (a) its content persists across signal/window/leadership churn AND (b) it is causally consumed by a decision or equation AND (c) it converts experience into transferable judgment (helps on accounts beyond the one it was learned from). Activity logs that merely record what happened belong in Tiers 1-5, not Tier 6.

META-PRINCIPLE — META-T6-002: STATE vs CHARACTER SEPARATION: The system distinguishes State (current, fluctuating — Tiers 1-5) from Character (stable behavioral priors — Tier 6). A snapshot entity (carries assessed_at/computed_at) is State; a prior that explains and predicts many snapshots is Character.

META-PRINCIPLE — META-T6-003: JUDGMENT-OVER-ACTIVITY PRINCIPLE: Where a Tier 6 estimate (DNA prior, narrative phase, regret score, dependency coefficient) conflicts with raw recent activity, the system surfaces the tension rather than silently trusting either. Estimated judgment never auto-overrides observed fact, and observed noise never silently erases an established prior.

The Unasked 5-Why: Why does the system need entities at all?

1st	To store information.
2nd	To make decisions.
3rd	To learn.
4th	To preserve institutional memory.
5th	To ACCUMULATE JUDGMENT.

Once the deepest purpose is accumulating judgment, every entity earns its place by a new test — and the entities that only pass THIS test, not the act-correctly-now test, are precisely Tier 6.

ENTITY 34: ORGANIZATIONAL_DNA

The persistent strategic personality of an account — risk appetite, vendor openness, decision velocity, governance style, innovation posture — that survives leadership changes, initiatives, and windows.

WHY #1 — Why does the system need Organizational_DNA when it already has Organizational_Assessment?

Because the model has State but not Character. Organizational_Assessment is a snapshot (it carries assessed_at) — a current reading. It has no home for the account's STABLE behavioral priors. SNB differs from Riyad Bank not by assets or signals but by a persistent personality: how it buys, how it governs, how aggressively it procures. Without DNA, the system re-derives that personality from scratch every cycle and re-learns the same account forever.

NEW RULE — DNA-RECORD-001: Each account carries one Organizational_DNA profile of stable behavioral priors (innovation_posture, risk_appetite, vendor_openness, procurement_aggressiveness, consensus_style, buying_archetype, transformation_velocity). DNA persists across leadership change, initiatives, and windows; it is updated slowly and deliberately, never per-signal.

WHY #2 — Why must DNA be separated from the Assessment snapshot rather than just kept as the latest snapshot?

Because they have opposite lifecycles and opposite jobs. DNA changes slowly (years); Assessment changes fast (a reorg, a new CTO). DNA is the PRIOR; Assessment is the current READING against that prior. Conflating them means a single turbulent quarter overwrites years of established character — destroying exactly the institutional memory DNA exists to protect.

NEW RULE — DNA-PRIOR-001: Organizational_DNA is the prior; Organizational_Assessment is the current reading. Assessments are interpreted AGAINST DNA. A sharp deviation between reading and prior is itself a signal (something is changing) — it does not silently overwrite the prior. DNA updates only when deviation persists across multiple assessments.

WHY #3 — Why does DNA change the way the system reasons and scores, not just what it stores?

Because behavior repeats while signals fluctuate. A procurement-aggressive, high-vendor-openness account converts differently from a consensus-bound, risk-averse one given the IDENTICAL signal. DNA conditions interpretation: it sets expected decision velocity (timing models), expected consensus difficulty (committee models), and expected displacement feasibility (entrenchment models). It is a conditioning input across the decision stack.

NEW RULE — DNA-CONDITIONING-001: Organizational_DNA conditions downstream models: it adjusts expected decision velocity in timing/window prediction, expected consensus difficulty in Buying_Committee modeling, and baseline vendor-displacement feasibility. Two accounts with identical signals but different DNA receive different strategies and pacing.

WHY #4 — Why is DNA transferable judgment rather than just per-account memory?

Because archetypes generalize. Once the system has characterized enough accounts, it can recognize a buying_archetype ('auditor-led, proof-demanding, slow-consensus' — the SNB pattern) in a NEW account from sparse early signals, and apply learned playbooks immediately. DNA turns one account's hard-won lessons into priors for similar accounts — passing the Tier 6 wiser-not-busier test.

NEW RULE — DNA-ARCHETYPE-001: DNA profiles cluster into reusable buying archetypes. A new or sparse account is assigned a provisional archetype from early signals, importing the archetype's learned playbook and expected behaviors until enough account-specific data refines its own DNA. Archetype assignment confidence is tracked and gates how strongly it drives strategy.

WHY #5 — Deepest reason Organizational_DNA exists?

Because the system's job is to accumulate judgment, not just state. DNA is where the system stops treating each account as a fresh stranger and starts knowing who it is dealing with. It is the difference between a system that reacts to the latest signal and one that understands an organization's nature — the foundation of strategic, rather than reactive, selling.

NEW RULE — DNA-PRIMACY-001: No account strategy may ignore established Organizational_DNA. Where a proposed action contradicts known character (e.g. fast-close tactics on a slow-consensus, governance-heavy account), the system flags the mismatch and requires justification or re-strategy before proceeding.

ORGANIZATIONAL_DNA — Field Specification

Field	Type	Business Rule	Source
dna_id	UUID	Immutable; one active per account	DNA-RECORD-001
account_id	UUID	FK to Account	—
innovation_posture	Enum	PIONEER / FAST_FOLLOWER / PRAGMATIST / LAGGARD	DNA-RECORD-001
risk_appetite / vendor_openness	Enum	Stable behavioral priors	DNA-RECORD-001
procurement_aggressiveness / consensus_style	Enum	How it buys / how it decides	DNA-CONDITIONING-001
buying_archetype	Enum	Reusable cluster (e.g. AUDITOR_LED_PROOF_DEMANDING)	DNA-ARCHETYPE-PE-001
transformation_velocity	Enum	Pace of change adoption	DNA-CONDITIONING-001
archetype_confidence	Int 0-100	Strength of provisional archetype fit	DNA-ARCHETYPE-PE-001
last_revised_at / stability_window	Timestamp	Slow, deliberate updates only	DNA-PRIOR-001

ENTITY 35: MARKET_NARRATIVE

A market-level story arc (Open Banking, AI Banking, Embedded Finance, Digital SME Lending) with an emergence-acceleration-peak-decline lifecycle. The first genuinely supra-account entity: it lives above accounts, not inside them.

WHY #1 — Why model narratives at all when the system already tracks signals, initiatives, and windows?

Because accounts don't buy when a signal score crosses a threshold — they buy when a narrative becomes dominant. Digital onboarding (2020), embedded finance (2022), open banking (2024), AI transformation (2025): budgets, mandates, and executive attention flow to whatever narrative is ascendant. The system tracks account intelligence but is blind to narrative intelligence — the actual current that carries buying behavior.

NEW RULE — NARR-RECORD-001: The system maintains Market_Narrative entities (e.g. OPEN_BANKING, AI_BANKING, BNPL, DIGITAL_SME_LENDING) tracking lifecycle phase (EMERGING / ACCELERATING / PEAK / DECLINING) with momentum and evidence sources. Narratives are market-level, not account-scoped.

WHY #2 — Why is Narrative the first entity that lives ABOVE the account, and why does that matter?

Because every entity in Tiers 1-5 is account-scoped — it describes one organization. Narrative is supra-account: one narrative simultaneously shapes SNB, SAB, Alinma, and the whole segment. This is why it feels like a category shift — it is the system's first model of the MARKET as an entity, not just a collection of accounts. It enables reasoning about the environment all accounts share.

NEW RULE — NARR-SCOPE-001: Market_Narrative is a shared, market-level entity referenced by many accounts simultaneously. It is the system's first-class model of the shared environment. Account-level entities link TO narratives; narratives never belong to a single account.

WHY #3 — Why must each account carry a Narrative Alignment Score?

Because accounts buy INSIDE narratives, not outside them. An account whose strategic priorities align with an accelerating narrative is far more buyable for a solution framed in that narrative than one misaligned or aligned to a declining narrative. Alignment is the bridge between market-level narrative and account-level opportunity.

NEW RULE — NARR-ALIGN-001: Each account computes a Narrative_Alignment_Score per active narrative (how well its initiatives, signals, and DNA align). High alignment to an ACCELERATING narrative raises framing relevance and feeds the opportunity assessment; alignment to a DECLINING narrative discounts it.

WHY #4 — Why does narrative phase change timing and messaging, not just scoring?

Because the same offer lands differently across the lifecycle. During EMERGING/ACCELERATING, education and vision sell; at PEAK, proof and differentiation sell (everyone is pitching it); during DECLINING, the narrative is a liability and framing must shift. Riding a narrative early is leadership; riding it at peak is commoditization. Phase conditions both WHEN and HOW to engage.

NEW RULE — NARR-PHASE-001: Narrative phase conditions outreach framing and timing: EMERGING/ACCELERATING favors vision/education framing and first-mover urgency; PEAK favors proof/differentiation against a crowded field; DECLINING triggers reframing away from the fading narrative. A Buying_Window opened by a signal intersecting an accelerating narrative carries a higher Window Multiplier than the same signal in a narrative vacuum.

WHY #5 — Deepest reason Market_Narrative exists?

Because it lets the system understand WHY a market is moving, not just that an account emitted a signal. It is judgment about the environment itself — transferable across every account in the segment, and predictive of where attention and budget will flow next. It is the system reading the weather, not just the thermometer.

NEW RULE — NARR-PRIMACY-001: Opportunity and timing reasoning must consult active Market_Narratives and the account's alignment. A signal is interpreted in the context of the prevailing narrative; the system distinguishes a signal that rides a rising narrative from an identical signal disconnected from any narrative.

MARKET_NARRATIVE — Field Specification

Field	Type	Business Rule	Source
narrative_id	UUID	Immutable; market-level	NARR-RECORD-001
narrative_name	Enum	OPEN_BANKING / AI_BANKING / BNPL / EMBEDDED_FINANCE / DIGITAL_SME_LENDING / ...	NARR-RECORD-001
lifecycle_phase	Enum	EMERGING / ACCELERATING / PEAK / DECLINING	NARR-RECORD-001
momentum / evidence_sources	Float/Array	Rate of spread + provenance	NARR-RECORD-001
scope	Const	Market-level; referenced by many accounts	NARR-SCOPE-001
phase_changed_at	Timestamp	Lifecycle transition tracking	NARR-PHASE-001

Companion field: Account-side companion field (added to ACCOUNT / ACCOUNT_STRATEGY):
narrative_alignment_scores : Map[narrative_id -> 0-100] per NARR-ALIGN-001.

ENTITY 36: COUNTERFACTUAL_RECORD

The system's model of regret: for a decision actually taken, the alternative decisions not taken, their predicted outcomes, the observed actual outcome, and a regret score — bounded by explicit confidence so estimates never masquerade as fact.

WHY #1 — Why does the system need counterfactuals when it already learns from outcomes?

Because outcome learning only knows what happened, never what could have happened. We engaged SNB and lost — would WAITING have won? Current architecture cannot answer. Mature decision systems learn from the deals they DIDN'T do and the timing they DIDN'T choose. Without counterfactuals the system optimizes only along paths it actually took, blind to better paths it never tried.

NEW RULE — CF-RECORD-001: For material decisions (engage/wait, channel choice, timing, connector path), a Counterfactual_Record stores the chosen decision, the principal alternatives not taken, each alternative's predicted outcome, the observed actual outcome, and a resulting regret_score.

WHY #2 — Why is the precision guardrail (confidence, no auto-drive) essential rather than optional?

Because a counterfactual is an ESTIMATE, not an observation. An estimated 'we would have won by waiting' is exactly the kind of confident fiction that the Section 1 hallucination and metric-manipulation failure modes warn against. If regret scores were treated as ground truth they could justify any narrative the model prefers. The guardrail keeps the entity honest.

NEW RULE — CF-CONFIDENCE-001: Every Counterfactual_Record carries an explicit confidence and is labeled ESTIMATE. Counterfactual regret may inform model recalibration and human review but may NEVER auto-drive a decision the way an observed Outcome_Attribution can. Low-confidence counterfactuals are advisory only.

WHY #3 — Why classify regret into false-positive / false-negative / mistiming types?

Because different errors demand different corrections. Premature engagement (false positive: acted when we should have waited) corrects the readiness/patience gate; missed engagement (false negative: waited when we should have acted) corrects window-closure urgency; right-decision-wrong-time corrects the timing model. A single scalar regret can't route the correction.

NEW RULE — CF-TAXONOMY-001: Regret is classified: FALSE_POSITIVE (premature engagement), FALSE_NEGATIVE (missed/over-delayed engagement), MISTIMING (right action, wrong moment), MISCHANNEL/MISPATH (right timing, wrong route). Each class routes correction to a specific model (readiness gate, window urgency, timing curve, path-finding).

WHY #4 — Why feed counterfactuals into the same feedback machinery as real outcomes, but separately weighted?

Because they ARE learning signal — just lower-grade than observed fact. They should recalibrate patience gates, timing curves, and decision patterns, but at a discounted weight reflecting their estimated nature, and always reversibly. This lets the system learn from roads not taken without overfitting to imagined results.

NEW RULE — CF-FEEDBACK-001: Counterfactual_Records produce Feedback_Loop_Entries at a confidence-discounted weight relative to observed outcomes. They may adjust patience gates, timing curves, and Decision_Pattern confidence within governed bounds; immutable meta-principles are never affected. All such adjustments are reversible.

WHY #5 — Deepest reason Counterfactual_Record exists?

Because judgment is built as much from imagined alternatives as from lived results — a strategist who only learns from what they did, never from what they might have done, plateaus. This entity lets the system develop a sense of the roads not taken: the foundation of genuinely advanced strategy and the clearest embodiment of accumulating judgment rather than activity.

NEW RULE — CF-PRIMACY-001: Material engage/wait and timing decisions should generate a Counterfactual_Record at outcome resolution. The system tracks aggregate regret by decision type as a core measure of judgment quality, distinct from raw win/loss.

COUNTERFACTUAL_RECORD — Field Specification

Field	Type	Business Rule	Source
counterfactual_id	UUID	Immutable	CF-RECORD-001
decision_ref / account_id	UUID	The decision evaluated	CF-RECORD-001
chosen_decision	Object	What was actually done	CF-RECORD-001
alternatives	Array[Object]	Roads not taken + predicted outcomes	CF-RECORD-001
observed_outcome	Object	What actually happened	CF-RECORD-001
regret_score / regret_class	Float/Enum	Magnitude + FALSE_POS/NEG/MISTIMING/MISPATH	CF-TAXONOMY-001
confidence / label	Int/Const	ESTIMATE; gates use	CF-CONFIDENCE-001
feedback_entry_id	UUID	Discounted-weight link	CF-FEEDBACK-001

ENTITY 37: MARKET_DEPENDENCY_GRAPH

The model of how one account's outcome changes the probability of others — reference, reputation, and network effects. The leap from resource allocation to genuine portfolio theory: opportunities are correlated, not independent.

WHY #1 — Why does the system need a dependency graph when it already has Portfolio_Allocation?

Because Portfolio_Allocation currently assumes opportunities are INDEPENDENT, and reality is correlated. Winning SNB changes the probability of Riyadh, SAB, and Alinma through reference effects, reputation effects, and network effects. Allocating capital as if each account is a separate bet ignores that the bets move together — leaving compounding reference value on the table.

NEW RULE — DEP-RECORD-001: The Market_Dependency_Graph stores directed dependency edges between accounts: how a defined outcome at account A (win, pilot, public reference) shifts win-probability or readiness at account B. Edges carry an effect type (REFERENCE / REPUTATION / NETWORK / COMPETITIVE) and a magnitude.

WHY #2 — Why does this turn allocation into actual portfolio theory rather than a nicer ranking?

Because portfolio theory optimizes over COVARIANCE between bets, not just individual expected values. A moderate-EROA account that unlocks three high-value references can beat a high-EROA isolated account. Dependency edges let Portfolio_Allocation value an account partly by what winning it does to the rest of the portfolio — sequencing 'keystone' accounts first.

NEW RULE — DEP-PORTFOLIO-001: Portfolio_Allocation consumes the dependency graph: an account's effective value includes its expected lift to dependent accounts (keystone value). Allocation may prioritize a keystone account over a higher-standalone-EROA account when downstream reference/network lift justifies it.

WHY #3 — Why must dependency coefficients degrade gracefully and start as assumptions?

Because the correlation magnitudes (+20% Riyadh, +15% SAB) are the hardest values in the entire spec to populate with real data early — there simply aren't enough wins yet to measure them. If treated as known, they become confident fiction driving real capital. They must begin as explicit assumptions and earn empirical weight over time.

NEW RULE — DEP-CONFIDENCE-001: Dependency magnitudes carry a confidence and a source (ASSUMED / ANALOG / EMPIRICAL). Early-stage edges are explicitly ASSUMED and capped in how strongly they influence allocation. Edges earn EMPIRICAL status only after observed reference/network effects, at which point their influence cap lifts.

WHY #4 — Why model competitive (negative) dependencies, not just positive reference lift?

Because outcomes can hurt as well as help. Losing publicly at one account, or being seen to fail, can depress others (negative reputation effect); and winning an incumbent's flagship can harden a competitor's defense elsewhere. A graph that only models positive lift is dangerously optimistic.

NEW RULE — DEP-NEGATIVE-001: Dependency edges may be negative: a public loss or failure can lower dependent accounts' probability (reputation contagion), and some wins trigger competitive hardening elsewhere. Allocation and Risk_Assessment consider negative dependencies; high negative blast-radius raises the risk weight of an action.

WHY #5 — Deepest reason Market_Dependency_Graph exists?

Because it lets the system reason about the market as a connected system rather than a list of isolated targets — understanding that its moves ripple. It is judgment about second-order effects, the most advanced form of strategic reasoning in the architecture, and the final piece that makes the system a portfolio strategist rather than an account-by-account optimizer.

NEW RULE — DEP-PRIMACY-001: Account sequencing and capital allocation must consider dependency effects where edges exist. The system does not treat accounts as independent when known reference, reputation, network, or competitive dependencies link them; isolated-bet allocation is permitted only where no dependency is known.

MARKET_DEPENDENCY_GRAPH — Field Specification

Field	Type	Business Rule	Source
dependency_id	UUID	Immutable directed edge	DEP-RECORD-001
from_account_id / to_account_id	UUID	A influences B	DEP-RECORD-001
effect_type	Enum	REFERENCE / REPUTATION / NETWORK / COMPETITIVE	DEP-RECORD-001
trigger_outcome	Enum	WIN / PILOT / PUBLIC_REFERENCE / LOSS / FAILURE	DEP-NEGATIVE-001
magnitude / direction	Float/Sign	Prob shift (may be negative)	DEP-NEGATIVE-001
confidence / source	Int/Enum	ASSUMED / ANALOG / EMPIRICAL	DEP-CONFIDENCE-001
influence_cap	Float	Limits sway while ASSUMED	DEP-CONFIDENCE-001

DESIGN DECISION — RELATIONSHIP_EDGE FLOW DECOMPOSITION (not a new entity)

The insight that relationships carry distinct trust, influence, information, and access flows is correct, but these four flows always share the same directed node-pair and edge lifecycle. Spawning a parallel Relationship_Flow entity would duplicate every node-pair and create a two-graph synchronization burden. The cleaner model decomposes the single scalar `strength` on the existing Relationship_Edge (Entity 13) into four typed flows. One graph, richer edges.

NEW RULE — EDGE-FLOW-001: Relationship_Edge.strength is decomposed into four independent typed flows on the same directed edge: trust_flow, influence_flow, information_flow, introduction(access)_flow. Each is scored and decays independently. The legacy single strength becomes a derived/weighted summary, never the source of truth.

NEW RULE — EDGE-FLOW-002: Trust and influence are explicitly NOT interchangeable: a contact may have high trust_flow but low influence_flow (likes us, no buying power) or the reverse (controls the decision, low rapport). Models consume the specific flow they need — committee/consensus reads influence_flow; warm-intro safety reads trust_flow.

NEW RULE — EDGE-PATH-001: Path-finding optimizes over the RELEVANT flow, not aggregate strength: shortest path is not best path. An introduction route maximizes the weakest link of (trust x introduction) flow along the path; an influence route to sway a committee maximizes influence_flow to the decision-maker. The system may select different paths to the same target depending on the goal.

Relationship_Edge — Added / Revised Fields

Field	Type	Business Rule	Source
trust_flow	Float 0-1	Confidence/rapport carried by edge	EDGE-FLOW-001
influence_flow	Float 0-1	Buying/decision sway carried by edge	EDGE-FLOW-001
information_flow	Float 0-1	Quality/candor of intel via edge	EDGE-FLOW-001
introduction_flow	Float 0-1	Willingness/ability to open access	EDGE-FLOW-001
strength (derived)	Float	Weighted summary of the four flows	EDGE-FLOW-001

REVISED ARCHITECTURE — 6 TIERS, 37 ENTITIES

Tier 6 changes the category of the specification. The entity count rises to 37 (33 + 4); Relationship_Edge is enriched rather than duplicated. The system now models not only State and Action but Character, Environment, Regret, and Inter-account Dependency.

Tier 1 — Core	5	Identity & events the system revolves around
Tier 2 — Intelligence	10	What the system knows about the world
Tier 3 — Decision	8	How the system decides what to do
Tier 4 — Governance	6	How the system governs itself
Tier 5 — Analytics	4	What the system learns (what happened)
Tier 6 — Strategic Memory	4	WHY reality behaves as it does (judgment)
TOTAL	37	

The dividing line: every entity in Tiers 1-5 can justify itself by "does it help us act correctly NOW?" Every Tier 6 entity justifies itself only by "does it help us judge better NEXT TIME, even on a different account?" That is the line between a system that optimizes actions and one that develops judgment.